

# Transformarea unui număr din baza 10 în baza 2

Informatică — Clasa a IX-a • Fișă de lucru

|                  |        |       |
|------------------|--------|-------|
| Nume și prenume: | Clasa: | Data: |
|------------------|--------|-------|

## 1. De ce baza 2?

Calculatoarele stochează toate informațiile folosind **biți** — cifre care pot fi doar 0 sau 1. De exemplu, litera A este stocată ca 01000001.

### Baza de numerație

Reprezintă numărul de simboluri distincte folosite pentru a scrie numerele:

Baza 10 (zecimală): 0–9 • Baza 2 (binară): 0, 1 • Baza 5: 0–4 • Baza 16 (hex): 0–9, A–F

### Baza 10 vs. Baza 2 — același număr

În baza 10, fiecare poziție este o putere a lui 10:

$$1\ 3\ 7 = 1 \cdot 10^2 + 3 \cdot 10^1 + 7 \cdot 10^0 = 100 + 30 + 7 = 137$$

În baza 2, fiecare poziție este o putere a lui 2:

$$1\ 0\ 0\ 0\ 1\ 0\ 0\ 1 = 128 + 8 + 1 = 137$$

| Număr  | Baza 10 | Baza 2   | Nr. de biți |
|--------|---------|----------|-------------|
| Șapte  | 7       | 111      | 3           |
| Zece   | 10      | 1010     | 4           |
| O sută | 100     | 1100100  | 7           |
| —      | 137     | 10001001 | 8           |

### Observație

Același număr are nevoie de mai multe cifre în baza 2, dar calculatorul lucrează natural cu 0 și 1: un circuit poate fi doar închis (1) sau deschis (0).

## 2. Algoritmul — Împărțiri repetate la 2

**De ce funcționează?** Restul împărțirii la 2 este exact ultima cifră binară:

- număr impar  $\rightarrow$  ultima cifră 1  $\rightarrow n \% 2 = 1$
- număr par  $\rightarrow$  ultima cifră 0  $\rightarrow n \% 2 = 0$

Împărțirea întreagă  $n // 2$  elimină acea cifră, exact cum  $237 // 10 = 23$  elimină ultima cifră zecimală. Repetăm până când numărul devine 0 și citim resturile **de jos în sus**.

**Exemplu: 13 în baza 2**

| Număr | $n \% 2$ (cifra binară)     | $n // 2$ (continuăm cu) |
|-------|-----------------------------|-------------------------|
| 13    | 1 $\leftarrow$ ultima cifră | 6                       |
| 6     | 0                           | 3                       |
| 3     | 1                           | 1                       |
| 1     | 1 $\leftarrow$ prima cifră  | 0 (stop)                |

Citim resturile de jos în sus: 1101

**13 (baza 10) = 1101 (baza 2)**

### Verificare

$$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1 = 13$$

### Pseudocod

```
citește n
rezultat ← ""
cât timp n > 0 execută
    rest ← n % 2
    rezultat ← str(rest) + rezultat    // cifra la stânga
    n ← n // 2
scrie rezultat
```

### Atenție

Dacă  $n = 0$ , rezultatul este direct "0".

### 3. Cod Python (referință)

---

#### Varianta pas cu pas

```
n = 13
rezultat = ""
while n > 0:
    rezultat = str(n % 2) + rezultat
    n = n // 2
print("13 în baza 2 =", rezultat)
```

#### Rezultat afișat de program:

13 în baza 2 = 1101

#### Funcție reutilizabilă

```
def in_baza_2(n):
    if n == 0:
        return "0"
    rezultat = ""
    while n > 0:
        rezultat = str(n % 2) + rezultat
        n = n // 2
    return rezultat

print(in_baza_2(13))
print(in_baza_2(42))
print(in_baza_2(255))
```

#### Rezultat afișat de program:

1101  
101010  
11111111

## 4. Conversia inversă: din baza 2 în baza 10

**Formulă:**  $N = b_k \cdot 2^k + \dots + b_1 \cdot 2^1 + b_0 \cdot 2^0$ , unde  $b_i$  sunt cifrele binare de la dreapta spre stânga.

```
s = "1101"
rezultat = 0
putere = 0
for cifra in reversed(s):
    rezultat += int(cifra) * (2 ** putere)
    putere += 1
print(s, "în baza 2 =", rezultat, "în baza 10")
```

### Rezultat afișat de program:

1101 în baza 2 = 13 în baza 10

### Metoda mai simplă — Schema lui Horner

Citim cifrele de la stânga la dreapta; la fiecare pas înmulțim rezultatul cu 2 și adăugăm cifra curentă.

**Exemplu: 1101**

| Cifra citită | Calcul          | Rezultat  |
|--------------|-----------------|-----------|
| 1            | $0 \cdot 2 + 1$ | 1         |
| 1            | $1 \cdot 2 + 1$ | 3         |
| 0            | $3 \cdot 2 + 0$ | 6         |
| 1            | $6 \cdot 2 + 1$ | <b>13</b> |

Obținem 13 — fără nicio putere calculată explicit!

## 5. Conexiuni cu lumea reală

| Aplicație         | Cum intervine baza 2                        |
|-------------------|---------------------------------------------|
| <b>Cod ASCII</b>  | Litera A = 65 = 01000001                    |
| <b>Culori RGB</b> | Fiecare componentă 0–255 → 8 biți → 1 octet |
| <b>Adrese IP</b>  | IPv4 are 4 octeți (32 biți)                 |
| <b>Fișiere</b>    | Orice fișier = secvență de 0 și 1 pe disc   |

### Curiozitate

Cu  $N$  biți poți reprezenta  $2^N$  valori distincte (de la 0 la  $2^N - 1$ ). Ex.: 8 biți → 256 valori (0...255).

## 6. Exerciții

### Exercițiul 1 — Transformare manuală

Transformă în baza 2 folosind tabelul cu împărțiri la 2 (resturile citite de jos în sus):

a) 7      b) 20      c) 64      d) 127

*Tabelul de împărțiri și rezultatul pentru fiecare:*

### Exercițiul 2 — Numărul de biți

Completează funcția care returnează numărul de biți necesari pentru `n`.

```
def numar_de_bit_i(n):  
    # TODO: returnează lungimea reprezentării în baza 2  
    ...  
  
# print(numar_de_bit_i(100))    # așteptat: 7  
# print(numar_de_bit_i(200))    # așteptat: 8
```

#### Indiciu

Numărul de biți = lungimea șirului returnat de `in_baza_2(n)`.

Scrie soluția:

### Exercițiul 3 — Conversie în baza b

Scrie un program care citește `n` și `b`, apoi afișează reprezentarea lui `n` în baza `b`.

**Restricții:**  $0 < b < 9$ ,  $0 < n < 100000$

#### Indiciu

Același algoritm ca la baza 2, dar înlocuiești 2 cu `b`.

Scrie codul:

#### Exercițiul 4 — Problemă: sumă de puteri ale lui 2

Se dă un număr natural  $n$ . Descompuneți-l ca sumă de puteri ale lui 2, afișate în ordine crescătoare, separate prin spațiu.

**Restricții:**  $1 \leq n < 2^{31}$

| Intrare | Ieșire   |
|---------|----------|
| 43      | 1 2 8 32 |

##### Indiciu

$$43 = 32 + 8 + 2 + 1 = 2^5 + 2^3 + 2^1 + 2^0$$

Legătura cu baza 2:  $43 = 101011_2$  — pozițiile cu cifra 1 dau puterile din sumă.

Scrie algoritmul / codul:

## Rezumat

| Concept                | Descriere                                                     |
|------------------------|---------------------------------------------------------------|
| <b>Algoritmul</b>      | Împărțiri repetate la 2; resturile citite de jos în sus       |
| <b>Operatori cheie</b> | % (rest) și // (câtul întreg)                                 |
| <b>Verificare</b>      | bin(n) returnează reprezentarea binară în Python              |
| <b>Invers</b>          | Suma cifra $\cdot 2^{\text{poziție}}$ , sau schema lui Horner |

### De reținut

Algoritmul funcționează pentru orice bază, nu doar 2 — schimbi doar 2 cu baza dorită!